

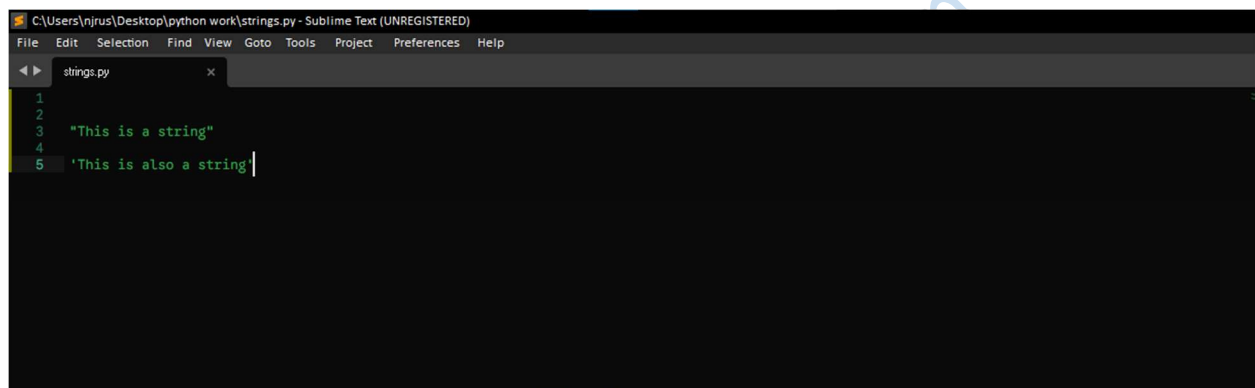
Data Types in Python

A **data type** tells python what kind of value we are working with. The main basic data types are:

- Strings
- Integers
- Floats
- Booleans

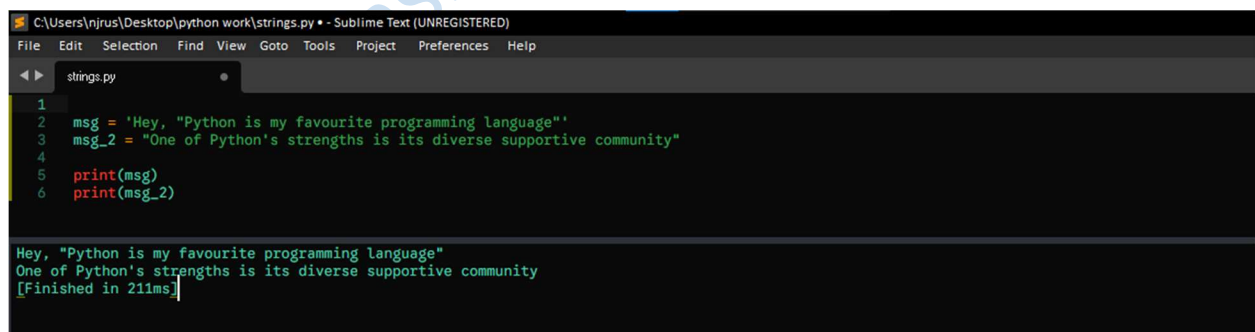
Strings

Strings are written inside quotation marks. Anything inside quotes is considered a string in python.



```
C:\Users\njrus\Desktop\python work\strings.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
strings.py
1
2
3 "This is a string"
4
5 'This is also a string'
```

Note that you can use both single or double quotes in your strings. This flexibility allows us to use apostrophes in our strings. Look Below:



```
C:\Users\njrus\Desktop\python work\strings.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
strings.py
1
2 msg = 'Hey, "Python is my favourite programming language"'
3 msg_2 = "One of Python's strengths is its diverse supportive community"
4
5 print(msg)
6 print(msg_2)

Hey, "Python is my favourite programming language"
One of Python's strengths is its diverse supportive community
[Finished in 211ms]
```

Some useful string methods

❖ Changing Case

One of the simplest tasks you can do with strings is changing the case of the words in a string.



FROM BASICS TO JUNIOR PYTHON DEV: Variables & Data Types (B)

```
C:\Users\njrus\Desktop\python work\name.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
name.py x
1
2 name = 'elon musk'
3
4 # name.title() changes every first letter of every work into uppercase
5 print(name.title())

Elon Musk
[Finished in 265ms]
```

The variable **name** refers to the value 'elon musk' which is a lowercase string. The method **title()** is an action that python can perform on a piece of data. The dot after name in **name.title()** tells python to make the title() method to act on the value associated with name. The title method changes each word to title case, where each word begins with a capital letter. Some other useful case methods are **upper()**, **capitalize()** & **lower()**. See below:

```
C:\Users\njrus\Desktop\python work\name.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
name.py x
1
2 name = 'elon musk'
3
4 # name.upper() changes every letter of every word to uppercase
5 print(name.upper())

ELON MUSK
[Finished in 234ms]
```

upper() changes every letter of every word to uppercase.

```
C:\Users\njrus\Desktop\python work\name.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
name.py x
1
2 name = 'elon musk'
3
4 # name.capitalize() changes only the first letter of a string to uppercase
5 print(name.capitalize())

Elon musk
[Finished in 231ms]
```

capitalize() changes only the first letter of a string to uppercase.

```
C:\Users\njrus\Desktop\python work\name.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
name.py x
1
2 name = 'ELON MUSK'
3
4 # name.lower() changes all letters in a string to lowercase
5 print(name.lower())

elon musk
[Finished in 221ms]
```



FROM BASICS TO JUNIOR PYTHON DEV: Variables & Data Types (B)

The **lower()** method is mostly used for storing data. Sometimes, you may not want to trust the capitalisation that your users provide. So, you will convert strings to lowercase before storing them. Later, when you want to display the information, you shall choose the case of best suit.

❖ Using variables in strings

In some situations, you'll want to use a variable's value inside a string. For example, you might want two variables to represent a first name & last name respectively and then want to combine those values to display someone's full name. You can do this in different ways:

Using f-strings

```
C:\Users\njrus\Desktop\python work\full_name.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
full_name.py x
1
2 first_name = 'moses'
3 last_name = 'kimani'
4
5 # -- method 1 (using f-strings)
6 full_name = f'{first_name} {last_name}'
7
8 print(full_name)

moses kimani
[Finished in 224ms]
```

Using format()

```
C:\Users\njrus\Desktop\python work\full_name.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
full_name.py x
1
2 first_name = 'moses'
3 last_name = 'kimani'
4
5 # -- method 1 (using format())
6 full_name = '{} {}'.format(first_name, last_name)
7
8 print(full_name)

moses kimani
[Finished in 227ms]
```

Using Comma-Separated Arguments

```
C:\Users\njrus\Desktop\python work\full_name.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
full_name.py x
1
2 first_name = 'moses'
3 last_name = 'kimani'
4
5 # -- method 1 (using comma-separated arguments)
6
7 print(first_name, last_name)

moses kimani
[Finished in 244ms]
```

Of all the three, I'd recommend you to stick with f-strings as they are very flexible to use. The f is for format and python formats the string by replacing the name of any variable in braces '{}' with its actual value.



FROM BASICS TO JUNIOR PYTHON DEV: Variables & Data Types (B)

You can do a lot with f-strings. For example, you could use them to compose complete messages using the information contained in variables. See below:

```
C:\Users\njrus\Desktop\python work\full_name.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
full_name.py x
1
2 first_name = 'moses'
3 last_name = 'kimani'
4
5 # -- using f-strings
6
7 print(f'Hello There! My first name is {first_name.title()} and my last name is {last_name.title()}')

Hello There! My first name is Moses and my last name is Kimani
[Finished in 207ms]
```

❖ Stripping whitespace in strings

A **whitespace** refers to any non-printing characters such as **spaces**, **tabs**, etc.

Extra whitespaces can be confusing to your programs. Consider these two strings; 'python' and 'python '. To a programmer, these two look pretty much the same. However, python detects the extra space in 'python ' and considers it significant unless you tell it otherwise.

It is important to consider these whitespaces because you'll often want to compare two strings to determine whether they are the same. A good example involves checking people's usernames when they register themselves in a website. Your goal may be to discourage the use of the same username by two people. Extra whitespaces can be confusing in these kinds of situations. Fortunately, python provides the **strip()** method which removes whitespaces from both sides of a string.

```
C:\Users\njrus\Desktop\python work\full_name.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
full_name.py x
1
2 name = 'moses '
3
4 print(name.strip())

moses
[Finished in 233ms]
```

This example does not clearly show the effect of strip() method on the string value. You will understand this better once you encounter a real-world problem (**SOON**).

Integers

An integer is a whole number. It can be negative, positive or zero.

You can perform operations on integers. These operations include; **addition**, **subtraction**, **multiplication**, **division** etc.



Floats

A **float** is any number with a decimal point. For example **-2.14, 3.0, 2.13** are all considered floats. You can perform operations on floats as well.

Boolean

A boolean represents one of two values; **True** or **False**. They are commonly used when making decisions.

```
6
7 age = 18
8
9 print(age > 20)
```

False
[Finished in 242ms]

Here, we checked if age is greater than 20. Python returns False implying that the age is not greater than the 20.

```
7
8 age = 18
9
10 print(age < 20)
```

True
[Finished in 228ms]

In this example, we checked if age is lesser than 20. Python returns True implying that the age is less than 20.

A practical example where we can use booleans is in a voting system. People who come to vote should at least be 18 years old and so we could design a program which filters out such people. **(To be covered under the topic of conditionals)**

BONUS TIP: You can usually check what type of data some variable represents if unsure. This can be done by using the `type()` function. See Below:

```
C:\Users\njrus\Desktop\python work\checking_type.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
checking_type.py x
1
2 name = 'Moses'
3 age = 32
4 height = 1.72
5 is_working = True
6
7 print(type(name))
8 print(type(age))
9 print(type(height))
10 print(type(is_working))
11
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
[Finished in 233ms]
```



Try Yourself

1. Use a variable to represent a person's name, and print a message to that person. Your message should be simple, such as, "Hello Moses, would you like to learn some Python today?"
2. Use a variable to represent a person's name, and then print that person's name in lowercase, uppercase, and title case.
3. Find a quote from a famous person you admire. Print the quote and the name of its author. Your output should look something like the following, including the quotation marks:

Albert Einstein once said, "A person who never made a mistake never tried anything new."

4. Repeat Exercise 3, but this time, represent the famous person's name using a variable called `famous_person`. Then compose your message and represent it with a new variable called `message`. Print your message.
5. Write addition, subtraction, multiplication, and division operations that each result in the number 8. Be sure to enclose your operations in `print()` calls to see the results. You should create four lines that look like this:
`print(5+3)`

Your output should simply be four lines with the number 8 appearing once on each line.

6. Use a variable to represent your favorite number. Then, using that variable, create a message that reveals your favorite number. Print that message.

TRY USING COMMENTS IN YOUR PROGRAMS WHERE POSSIBLE.

